Softwaredesign

Software Engineering II

S.A.F.T. Corp.

Secret Audio File Transformer

DHBW

DHBW Mannheim
29. Januar 2026

Änderungshistorie

Revision	Datum	Autoren	Beschreibung
1.0	29.01.2026	P	A: 1 - 2.2
1.1	29.01.2026	G	A: 3,5
1.2	29.01.2026	R	A: 4

Abkürzungen: Hinzugefügt/Added (A), Änderung/Changed (C), Löschung/Deleted (D),
Gerrit (G), Raphael (R), Tu (T), Paul (P), Johannes (J)

Inhaltsverzeichnis

1 Einleitung----------1
2 Benutzeroberfläche---------- 2
2.1 Corporate Design---------- 2
2.2 GUI Design---------- 3
3. Grafische Darstellungen----------4
3.1 Import---------- 4
3.2 Rendering und Editing---------- 5
3.3 Export---------- 7
4. Designentscheidungen---------- 8
4.1 Rendering nur bei Interaktion---------- 8
4.2 Viewport Rendering---------- 8
4.5 Spektrogramm Persistenz---------- 8
5. Zusammenfassung----------9

1 Einleitung

In diesem Dokument wird das Softwaredesign des Secret Audio File Transformers (S.A.F.T.) beschrieben. Der Fokus liegt auf der Gestaltung der Benutzeroberfläche (Corporate Design und GUI für mobile Endgeräte) sowie auf dem Entwurf der zentralen Komponenten und Services auf Klassen- und Ablaufebene, die mithilfe von Sequenzdiagrammen erläutert werden. Darüber hinaus wird dargestellt, wie die wichtigsten nicht-funktionalen Anforderungen durch konkrete Designentscheidungen im System umgesetzt werden.

2 Benutzeroberfläche

2.1 Corporate Design

Entsprechend des orangefarbenen Logos von S.A.F.T. wurde die folgende 4-teilige Farbpalette erstellt, um eine oberflächliche Zuordnung sämtlicher Designs zu der S.A.F.T. Corp. zu ermöglichen. Als Tool zur Generierung dieser Palette wurde coolors.co verwendet.

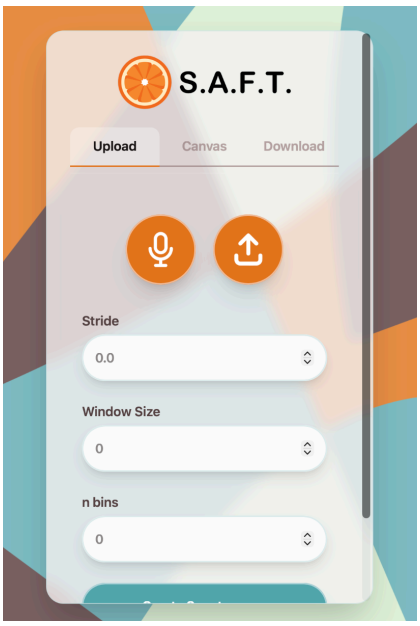
Saft-Main #e3761c	Saft-Mint #cfe3b5	Saft-Blue #50a7af	Saft-Brown #8e7172

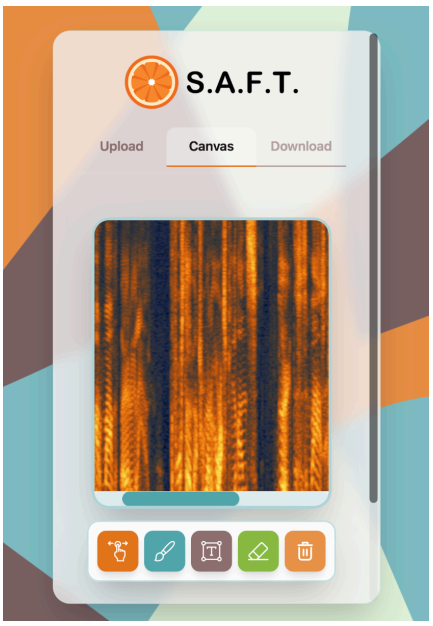
Ein damit mögliches Design ist:

Um ein ansprechendes und modernes Design zu wahren, sind außerdem alle Elemente von Benutzeroberflächen abgerundet oder rund.

2.2 GUI Design

Der S.A.F.T. ist primär als Applikation für Smartphones gedacht. Daher ist die gesamte GUI entsprechend eines schmalen und hohen Bildschirms orientiert und die Bedienelemente in der Größe für die Touch-Steuerung geeignet. Außerdem ist die Benutzeroberfläche entsprechend des Corporate Designs ansprechend und modern gestaltet. Das Beispiel-Design aus Abschnitt 2.1 wurde zusätzlich als Hintergrundbild ausgewählt. Buttons besitzen bei Betätigung eine “Klick”-Animation, welche diese kurz um 10% verkleinert. Die Oberflächen sehen vorerst wie folgt aus:

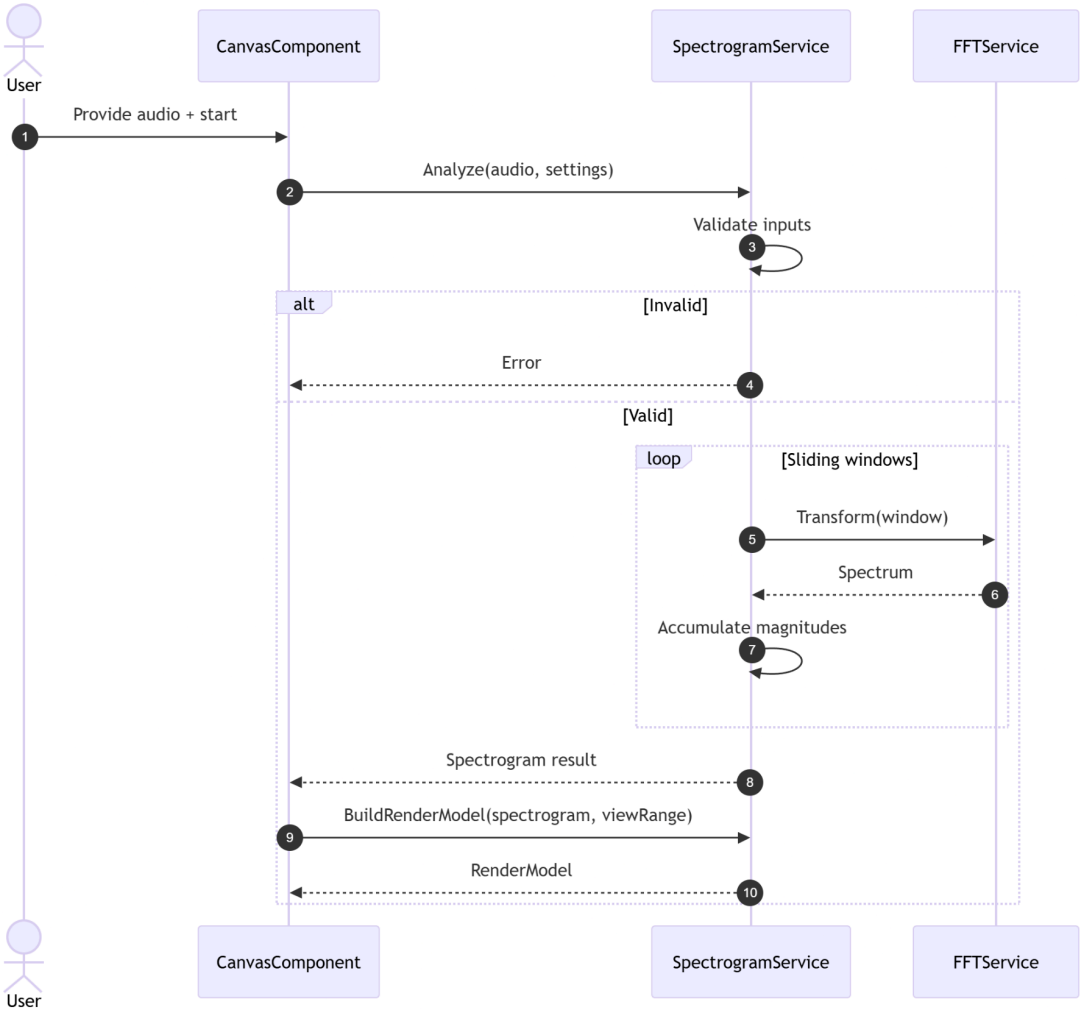




3. Grafische Darstellungen

3.1 Import

Das folgende Sequenzdiagramm zeigt, wie der Nutzer Audio startet und die CanvasComponent die Analyse beim SpectrogramService anstößt. Der SpectrogramService validiert die Eingaben und berechnet bei gültigen Daten in einer Sliding-Window-Schleife über den FFTService pro Fenster ein Spektrum, dessen Magnituden zum Spektrogramm akkumuliert werden. Anschließend liefert er das Spektrogramm an die CanvasComponent zurück und erzeugt auf Anfrage mit “BuildRenderModel(spectrogram, viewRange)” ein fürs Rendering optimiertes “RenderModel”. Bei ungültigen Eingaben wird statt der Berechnung ein Fehler an die CanvasComponent zurückgegeben.
Der entsprechende Mermaid-Code ist im Repository hinterlegt (hier).



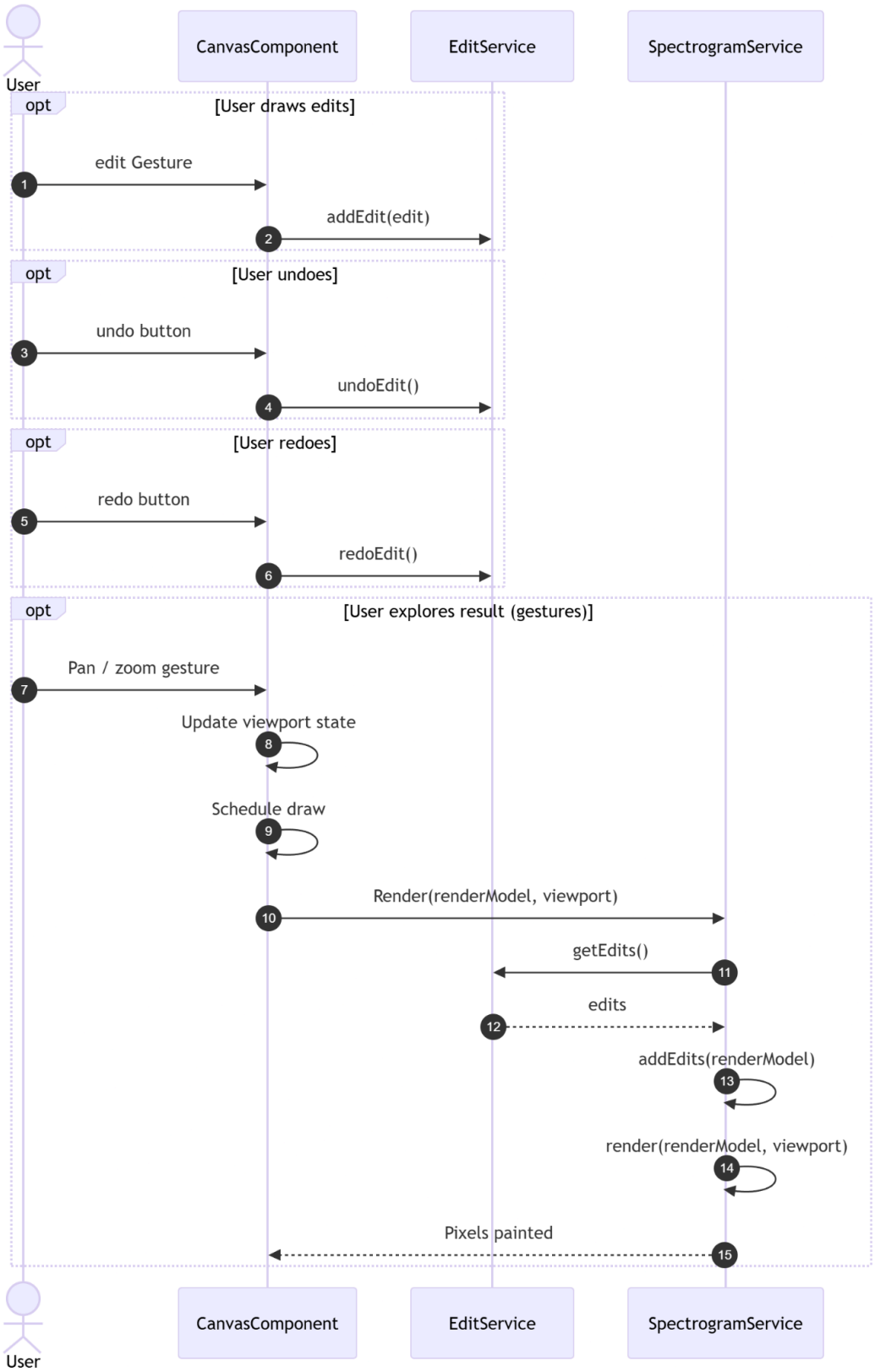
generiert auf: mermaid.live

Die Aufteilung in CanvasComponent, SpectrogramService und FFTService folgt dem Single-Responsibility-Prinzip: Die UI steuert nur den Ablauf, der SpectrogramService koordiniert die Domänenlogik, und der FFTService kapselt reine Signalverarbeitung. Durch die klare Schichtung (Separation of Concerns) werden Wartbarkeit und Testbarkeit besser, weil Validierung, Spektrogramm-Berechnung und FFT getrennt getestet und ausgetauscht werden können. Die frühe Validierung im SpectrogramService setzt „Fail fast“ um und verhindert unnötige Rechenarbeit bei fehlerhaften Eingaben.
Das zusätzliche BuildRenderModel(spectrogram, viewRange) unterstützt Performance und Flexibilität, weil Rendering-spezifische Aufbereitung vom Analyseergebnis entkoppelt bleibt und z. B. Viewport/Zoom ohne Neuberechnung der FFT bedient werden kann.

3.2 Rendering und Editing

Das Sequenzdiagramm auf der nächsten Seite zeigt, wie der Nutzer über die Canvas-Component Bearbeitungen ausführt und diese im EditService als Edits gespeichert werden. Für Undo/Redo triggert die CanvasComponent jeweils undoEdit() bzw. redoEdit() im EditService, der die Edit- Historie entsprechend zurück- oder vorwärtssetzt. Beim Erkunden des Ergebnisses (Pan/Zoom) aktualisiert die CanvasComponent den Viewport und plant ein Neuzeichnen, woraufhin der SpectrogramService mit Render(renderModel, viewport) aufgerufen wird. Der SpectrogramService holt dazu die aktuellen Edits via getEdits(), wendet sie auf das RenderModel an und rendert anschließend die Pixel zurück an die CanvasComponent.
Der entsprechende Mermaid-Code ist im Repository hinterlegt (hier).

Die Aufteilung in CanvasComponent, EditService und SpectrogramService folgt dem Single-Responsibility-Prinzip: UI/Interaktion, Edit-Verwaltung (inkl. Undo/Redo) und Rendering/ Domainlogik bleiben getrennt, was Änderungen risikoärmer macht. Durch klar definierte Service-APIs entsteht eine saubere Schichtenarchitektur (Separation of Concerns), die Wartbarkeit und Testbarkeit erhöht, weil Edit-Historie und Rendering unabhängig geprüft werden können. Außerdem unterstützt die Edit-Logik das Command-Pattern (Edits als Aktionen), wodurch Undo/Redo konsistent und erweiterbar wird, ohne die Rendering-Pipeline umzubauen. Schließlich bleibt das Rendering performant, weil Pan/Zoom nur den Viewport aktualisiert und gezielt ein Redraw auslöst, statt unnötig die gesamte Verarbeitung neu zu starten..

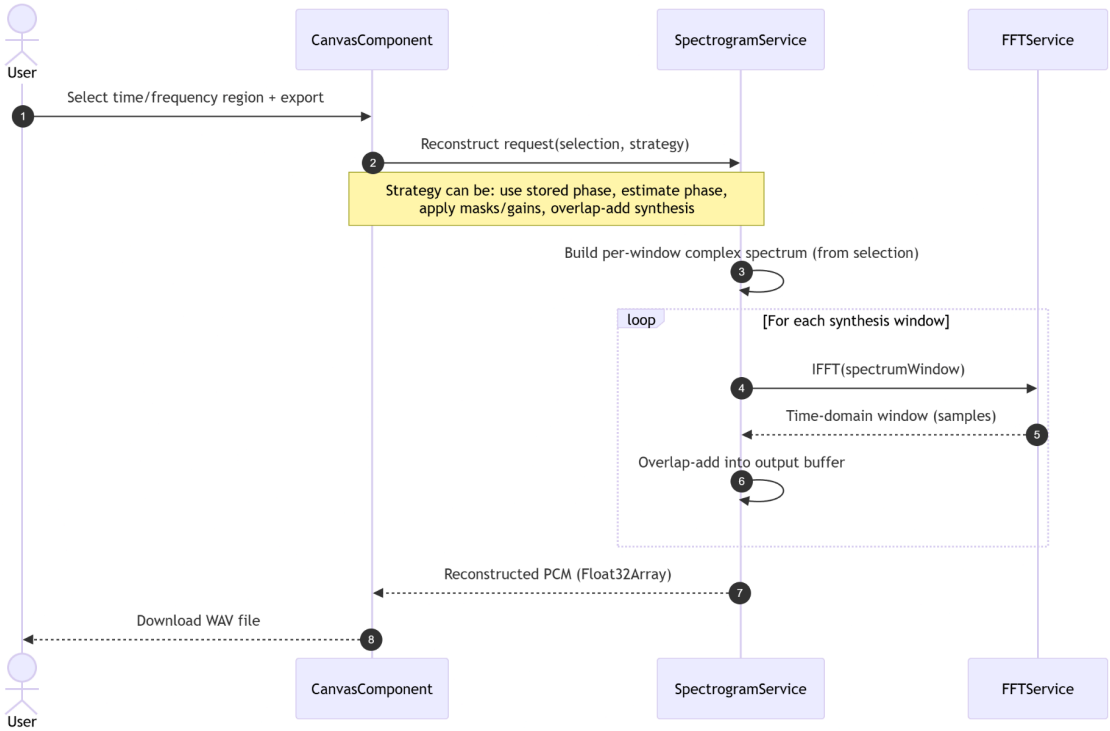


generiert auf: mermaid.live

3.3 Export

Das folgende Sequenzdiagramm zeigt, wie der Nutzer einen Zeit-/Frequenzbereich auswählt und in der CanvasComponent einen Export startet. Die CanvasComponent sendet dazu eine Reconstruct-Anfrage an den SpectrogramService, inklusive Auswahl und einer Strategie (z. B. gespeicherte Phase nutzen, Phase schätzen, Masken/Gains anwenden, Overlap-Add -Synthese). Der SpectrogramService baut daraus fensterweise komplexe Spektren und ruft pro Synthese-Fenster beim FFTService eine IFFT auf, um Zeitbereichs-Samples zu erhalten, die anschließend per Overlap-Add in einen Output-Buffer gemischt werden. Am Ende liefert der Service rekonstruiertes PCM (Float32Array) zurück, und die CanvasComponent stellt daraus eine WAV-Datei zum Download bereit.
Der entsprechende Mermaid-Code ist im Repository hinterlegt (hier).

Die Aufteilung in CanvasComponent, SpectrogramService und FFTService folgt dem Single-Responsibility-Prinzip: UI/Exportfluss, Rekonstruktionslogik und IFFT-Berechnung sind klar getrennt. Die Strategie-Übergabe ist ein Beispiel für das Strategy-Pattern und erhöht die Erweiterbarkeit, weil neue Rekonstruktionsverfahren ergänzt werden können, ohne den UI-Code oder die IFFT-Schicht zu ändern. Die fensterbasierte Verarbeitung mit Overlap-Add ist modular und gut testbar, da jedes Synthese-Fenster deterministisch verarbeitet werden kann. Gleichzeitig unterstützt die Kapselung des PCM-Outputs eine saubere Schnittstelle zur Dateierzeugung (WAV ggf mp3), sodass Rendering/Analyse und Dateiformatierung entkoppelt bleiben.



generiert auf: mermaid.live

4. Designentscheidungen

4.1 Rendering nur bei Interaktion

Wir haben mehrere Performance Maßnahmen eingebaut, damit sich Zoom, Pinch und Pan im Canvas flüssig anfühlen. Neu gerendert wird nur dann, wenn sich der View wirklich bewegt, und das Rendering läuft synchron zum Browser Takt über requestAnimationFrame.

4.2 Viewport Rendering

Wir rendern nicht das komplette Spektrogramm, sondern nur den Bereich, den der Nutzer gerade sieht. Dadurch bleibt das UI auch bei großen Dateien schnell.

4.3 FFT ohne UI Blockieren

Die FFT Berechnung ist so organisiert, dass die Oberfläche nicht blockiert. Die Arbeit wird in den Ablauf mit requestAnimationFrame integriert, damit Eingaben weiter reagieren.

4.4 Edit Verlauf für Undo und Redo

Für Edits nutzen wir eine separate Datenstruktur statt das Spektrogramm sofort zu überschreiben. Wir speichern die letzten zehn Schritte für Undo und Redo und schreiben sie danach gesammelt ins Spektrogramm.

4.5 Spektrogramm Persistenz

Das Spektrogramm wird in einer Datenbank gespeichert, damit es wieder geladen werden kann und nicht jedes Mal komplett neu aufgebaut werden muss.

4.6 Einfacher Export über bestehende Apps

Beim Export halten wir es bewusst einfach. Dateien sollen über vorhandene externe Chat Apps oder Share Funktionen weitergegeben werden, statt dass wir dafür eigene Features bauen.

4.7 Clientseitige Verarbeitung mit Web APIs

Die Berechnung passiert clientseitig über Web APIs und nicht auf einem Server. Das spart Backend Aufwand und vermeidet, dass Audiodateien hochgeladen werden müssen.

5. Zusammenfassung

Im Rahmen des Softwaredesigns werden die nicht-funktionalen Anforderungen gezielt durch konkrete Designentscheidungen und Strukturierungsmaßnahmen adressiert. Performance wird durch Viewport-Rendering (nur sichtbarer Bereich), Rendering nur bei Interaktion sowie eine nicht blockierende FFT-Verarbeitung unterstützt, sodass die UI reaktionsfähig bleibt (Eingaben ≤ 1 s) und Verarbeitung/Export in den geforderten Zeitgrenzen (je ≤ 10 s bei bis zu 5 Minuten Audio) realistisch erreichbar ist. Usability wird durch eine intuitive, mobile-optimierte und responsive GUI mit klaren Eingabefeldern, verständlichen Fehlermeldungen sowie PWA -Eigenschaften (installierbar, offline nutzbar) abgedeckt; zusätzlich wird die Nutzbarkeit in modernen Desktop- und Mobile-Browsern berücksichtigt.

Zuverlässigkeit wird durch frühe Validierung (Fail-fast), robuste Fehlerbehandlung bei ungültigen Parametern/defekten Dateien und eine Verarbeitungskette ohne Datenverlust bei Audio↔ Spektrogramm gestützt; außerdem bleibt die Audioausgabe nach Einbettungen funktionsfähig und abspielbar. Sicherheit wird durch clientseitige Verarbeitung, Verzicht auf dauerhafte Speicherung von Audio/temporären Daten (automatische Löschung nach Sitzung), keine Speicherung personenbezogener Daten sowie verschlüsselten Transport (HTTPS) erfüllt. Erweiterbarkeit ergibt sich aus der modularen Struktur (getrennte UI-Komponenten, Services, klare Schnittstellen) und dem Einsatz erweiterungsfreundlicher Muster (z. B. Strategy für Rekonstruktion, Command für Undo/Redo), sodass neue Editierfunktionen ohne grundlegende Umstrukturierung ergänzt werden können; eine nachvollziehbare, dokumentierte Code-Struktur unterstützt dies zusätzlich. Kompatibilität wird durch die Auslegung als plattformunabhängige Web/PWA-Anwendung (Desktop & Mobile) sowie die Unterstützung der definierten Import-/Exportformate (.wav, .mp3 indirekt durch PCM) sichergestellt.